



CAVY

Software Report

CODEBASE / FRAMEWORKS / CODE QUALITY / NFRs

Where AI helps in a coding lab, the trail of how it was used is kept.

Built, tested and measured by students, for classrooms.

9,825

lines of Python

457

automated tests

89%

test coverage

0 / 0 / 0

lint / type / security

Group 04

- Viral Kariya
- Vanshika Ahuja
- Yashika Parmar
- Carol Maria

0

At a glance

What was checked and what came out

CAVY lets students do coding labs with an AI assistant available, and keeps a structured record of how that help was used. This report describes how the software is built, why each framework was chosen, how its quality was checked (with the actual numbers) and how it meets the non-functional requirements. Every figure was produced by running the tool against the current source tree when the report was generated.

9,825 lines of Python in 44 files	5,467 lines of JavaScript (app + admin panel)	457 tests passing, 7,158 lines of tests	89.0% line + branch coverage (floor 70%)
0 Ruff issues in src and tests	0 mypy --strict errors	0 Bandit security findings	83% mutation score, scoring engine

The five things worth knowing

- **It is built as clear parts around one event log.** A single bridge class (CavyApi) is the only door between the screens and the logic, so the app, the admin panel and the tests all use the same code.
- **Every tool is clean.** Ruff and mypy strict report 0 issues, Bandit reports 0 findings across 9,569 lines, and pip-audit finds no known vulnerability in the 151 pinned packages.
- **The tests check results, not just execution.** 89% coverage, and mutation testing raised the scoring engine from 65% to 83% by fixing tests that were too forgiving.
- **Speed was measured, not guessed.** 30 students working at the same time: about 800 calls a second, none failed. The same test found a real bug (a database pool limit) that was then fixed.
- **The gaps are listed.** One very large file (api/bridge.py), plain HTTP on the classroom network, unsigned installers, and a scoring model validated on constructed cases, not yet on a real class.

1

Codebase

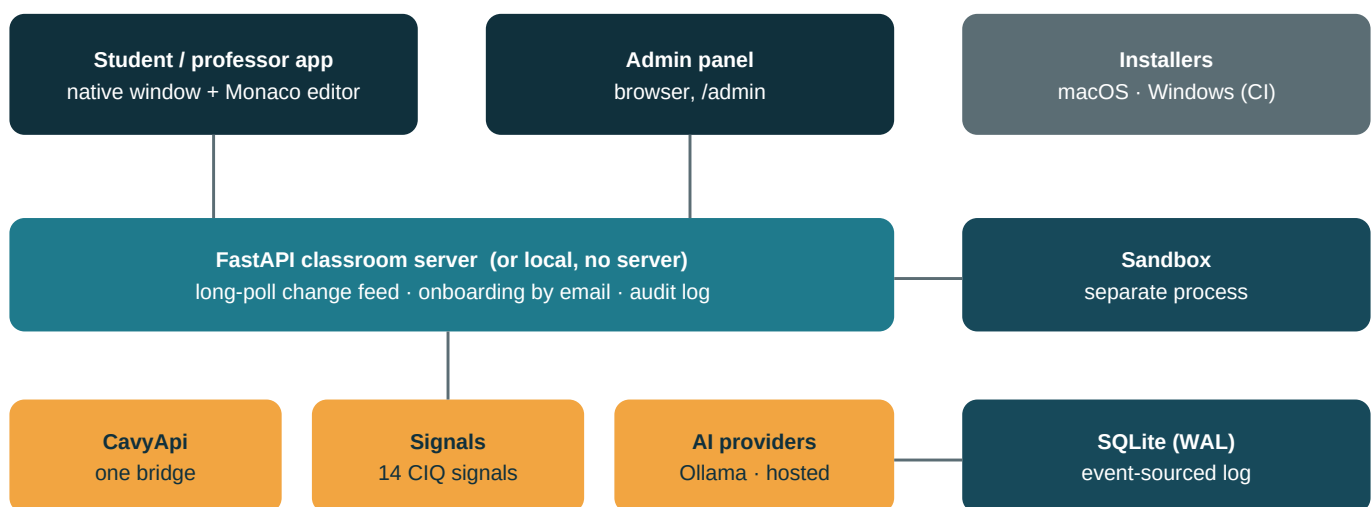
What was built, how it is organised, how to run it

1.1 What CAVY is

CAVY runs programming labs where an AI assistant is available throughout, and treats the assistant as part of the system instead of a black box: every prompt, reply, code edit, run and submission is stored as an event in the database the rest of the platform reads. The CIQ score, the progress charts and the professor's reports are all derived from that log.

It runs in two ways from the same code: standalone on one computer (its own SQLite file), or against a central classroom server that a professor and many students on one network share live, with a browser admin panel.

1.2 Architecture



1.3 Modules

Module	Lines	Responsibility
db/	2,615	SQLAlchemy models, the engine (WAL, connection pool), accounts, courses, resources, approvals
server/	2,111	FastAPI classroom server, onboarding by email, the browser admin panel
api/	1,982	CavyApi : the one bridge every screen talks to
client/	872	The app's side: local data or a server, lab-mode lock-down, remote calls
ai/	777	AIProvider interface; Ollama and hosted providers (OpenAI, Claude, Gemini, Grok, Groq)
signals/	670	The 14 CIQ signals computed from the event log
events/	141	Background, batched, append-only event logger
sandbox/	138	Runs student code in a separate process with a time limit

1.4 How the parts connect

The window (HTML/CSS/JS in the system web view, Monaco editor) calls one Python object, **CavyApi**, for everything. On a server the same class is reused unchanged: the server creates one per signed-in person and exposes its methods over HTTP. A change feed (long polling) tells open screens when something they show has changed, so a professor's upload appears on a student's screen without a refresh.

Students' code never runs inside the app: each run is a separate process with a time limit and, where the operating system supports it, CPU and memory limits. The event logger writes to the database on its own thread, so typing never waits for the disk.

1.5 Reproducible setup

`requirements.lock` pins 151 packages to the exact versions the tests run against; `pyproject.toml` declares the ranges and the optional groups (server, build, dev). The README covers setup, running, the quality gate, building installers and the architecture; `docs/` holds the classroom guide, Windows testing notes, performance results and the roadmap.

2

Frameworks

What was chosen, why, and what else was considered

Need	Chosen	Why this one	Alternatives weighed
Desktop shell	pywebview	Uses the system's web view (WebKit / WebView2), so the installer is about 70-80 MB, not an Electron-sized bundle; a plain Python object can be called straight from JavaScript.	Electron (bigger, adds Node), Tauri (needs Rust), PySide6 (our first version; slower to build and style).
Server	FastAPI + uvicorn	Typed request models, easy testing with TestClient, small footprint, serves the admin panel's static files.	Django (heavier, would duplicate the CavyApi layer), Flask (no built-in validation).
Database	SQLite (WAL) + SQLAlchemy 2	Zero setup for a classroom server or a laptop; WAL lets the event thread write while requests read; SQLAlchemy keeps the schema in typed models and could move to Postgres.	PostgreSQL (extra install, overkill for one class), plain sqlite3 (no model or migration help).
Code editor	Monaco	The editor VS Code uses: Python highlighting, indentation, find; a plain-textarea fallback keeps the app working without it.	CodeMirror (lighter, fewer features), a bare textarea (no indenting).
AI access	httpx + a provider interface	One small HTTP client; each company is a short class behind the same interface, so a new provider is about 30 lines and tests use a mock transport.	Each company's own SDK (4 extra dependencies that change often), LangChain (far heavier).
Installers	PyInstaller, Inno Setup, dmgbuild, GitHub Actions	One recipe builds each OS's app; Actions builds and smoke-tests both installers on clean machines.	py2app / briefcase (one OS or less mature), WiX (heavier than Inno Setup).
Quality tools	Ruff, mypy strict, pytest, Radon, Bandit, pip-audit, mutmut	Each covers one risk (next section).	Flake8 + isort + pyupgrade (Ruff replaces all three in one fast tool).

Small on purpose

No front-end build step, no ORM add-ons, no task queue, no message broker. The lock file lists what is actually installed and pip-audit checks it, so a package nobody uses would show up as a line nobody can explain.

3

Code quality

Nine checks, what each one means, and what it found

A different tool covers each kind of risk. Together they answer: is the style consistent, are the types right, is the logic simple enough to change safely, do the tests actually catch mistakes, is the code safe, and are the libraries safe. Each result below comes from running the command shown on the current code.

3.1 Ruff: style and likely bugs

What it checks. Ruff is a fast linter that replaces Flake8, isort and pyupgrade. It reads the code without running it and flags unused variables, wrong imports, risky patterns and style drift, so a reviewer does not have to.

```
ruff check src tests ; ruff format --check .
```

0	10	100	1
issues found	rule families on	line length limit	file with relaxed rules*

Rule family	What it catches
E, W	pycodestyle: layout, spacing, line length
F	Pyflakes: undefined names, unused imports and variables
I	isort: import order
UP	pyupgrade: old idioms that have a modern form (Python 3.11)
B	bugbear: likely bugs (mutable defaults, unused loop variables, bare excepts)
SIM	simplify: needless complexity
C4	comprehensions that can be simpler
N	PEP 8 naming
RUF	Ruff's own checks

Reading the result. Zero issues across source and tests, with the rules above all switched on. *The only relaxed rules are line length (and one naming rule) in the report generator and some validation scripts, whose lines are mostly prose. Nothing else is switched off.

3.2 mypy (strict): are the types right?

What it checks. mypy follows the type hints through the whole program and refuses code where a value could be the wrong type or *None* when the next line assumes it is not. Strict mode also requires every function to be annotated, which is what makes the check meaningful.

```
mypy src
```

0	?	strict	pre-commit
errors	source files checked	mode	runs it on every commit

Reading the result. The two bridge-style layers (CavyApi and the client that forwards to the server) pass plain dictionaries around, which is exactly where a typo in a key would hide in a demo. Strict typing catches that class of mistake before the code runs.

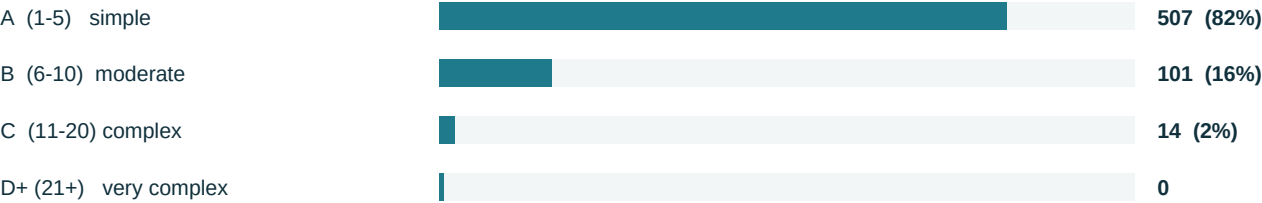
3.3 Radon: how complex is each function?

What it checks. Cyclomatic complexity counts the independent paths through a function (every *if*, loop and exception adds one). More paths means more tests are needed and more places for a bug to hide. Radon grades blocks A (1-5,

simple), B (6-10), C (11-20), D and worse (21+).

```
radon cc src/eaal_platform -a -s
```

622 blocks analysed (functions, methods, classes)	A (3.23) average complexity	14 blocks graded C	0 blocks graded D or worse
---	---	--	--



The five most complex blocks:

Function	File	Complexity
_apply	db/resources.py	13 (C)
get_student_submission	api/bridge.py	13 (C)
build_progress	progress_report.py	12 (C)
set_professor_courses	db/academics.py	12 (C)
assign_by_groups	db/academics.py	12 (C)

Reading the result. Nine in ten blocks are simple, and nothing is worse than moderately complex. The busiest functions are the ones that have to decide between many cases: reading a spreadsheet of students, validating a resource, setting up an account. During this work, several blocks that had grown past 20 were split into small helper functions, which is why none is now above 13.

3.4 Radon: maintainability, size and Halstead

What it checks. The maintainability index (0-100) combines size, complexity and comments into one score per file (A is easy to change, C is hard). Raw metrics describe the size; Halstead metrics estimate the effort to read and the number of bugs a program of this size tends to hide.

```
radon mi src/eaal_platform -s ; radon raw ... ; radon hal ...
```

41 / 44 files graded A	8,562 source lines of code (logical + physical, no blanks)	10% comment lines (plus docstrings)	10 Halstead predicted bugs (rough)
--	--	---	--

File (lowest three)	Maintainability index	Grade
api/bridge.py	0.0	C
db/academics.py	16.0	B
server/admin.py	18.5	B

Reading the result. Most files are grade A. api/bridge.py scores low purely because of its size (about 2,000 lines in one class); the code in it is simple, but one class that large is hard to hold in your head. Splitting it into modules is the first item in *Next steps*. Halstead's predicted-bugs figure is a rule of thumb (volume divided by 3,000), not a defect count; it is reported only to show the size of the code base.

3.5 pytest and coverage: do the tests exercise the code?

What it checks. pytest runs the automated tests. Coverage records which lines (and which branches of each *if*) the tests actually reached. A coverage floor of 70% is set in the project configuration, so a drop fails the run instead of just changing a number in a report.

```
pytest --cov=eaal_platform --cov-branch
```

457	1	89.0%	70%
tests passed	skipped (slow marker)	line + branch coverage	floor that fails the run

Coverage by module (the larger modules):

db/		96% (1430 stmts)
server/		95% (1274 stmts)
api/		94% (940 stmts)
client/		66% (609 stmts)
ai/		95% (462 stmts)
signals/		98% (330 stmts)
events/		98% (91 stmts)
mailer.py		85% (87 stmts)
progress_report.py		100% (63 stmts)

Reading the result. Business logic (accounts, courses, scoring, sandbox, event logger, server) is well covered. The lower bars are code that needs a real screen: the window set-up in `app.py` and the operating-system lock-down for lab mode, which can only be exercised by running the app on macOS and Windows. Tests include end-to-end HTTP tests of the server and two tests that keep the database pool large enough for a class.

3.6 mutmut: would the tests notice a bug?

What it checks. Coverage says a line ran, not that a test would fail if the line were wrong. mutmut makes small deliberate mistakes (turns a `<` into `<=`, a `0.5` into `0.6`, drops a condition) and re-runs the tests. If a test fails, the mutant is *killed*; if everything still passes, it *survived* and the tests are too forgiving at that spot. We scoped it to the scoring engine (`signals/compute.py`), the one place a silently wrong number would be hardest to notice.

```
mutmut run (scoped by [tool.mutmut] in pyproject.toml)
```

1,051	876	175	83.3%
mutants tried	killed (tests caught them)	survived	mutation score

First run		65.3% (686 of 1,051)
After the new tests		83.3% (876 of 1051)

Reading the result. The first run killed 65% of the mutants: the suite ran most lines but checked the exact values too loosely. Reading what survived showed the gaps, such as the exact reason a signal gives when it cannot be computed, the boundaries where a score is capped, and the evidence a signal reports. A set of exact-value tests was added for those cases and the score rose to 83%. The remaining 175 survivors are mostly equivalent changes (for example a different default message nobody reads) and the longitudinal comparison, listed as future work.

3.7 Bandit: security patterns in the code

What it checks. Bandit scans the source for patterns that are often security problems: running shell commands, hard-coded passwords, binding to every network address, weak hashes, loading untrusted data.

```
bandit -r src
```

0	0	0	9,569
high severity	medium severity	low severity	lines scanned

Reading the result. 0 findings. A few patterns are real by design and are accepted individually with a comment explaining why: the server listens on all network addresses because it is meant to be reached from other computers in the lab; the sandbox starts a subprocess with a fixed argument list (no shell); and a few strings that look like passwords are only field names. Each of the 18 suppressed checks sits next to its reason in the code, instead of being switched off globally.

3.8 pip-audit: are the libraries safe?

What it checks. The code can be perfect and still be unsafe if a library it uses has a known vulnerability. pip-audit compares every pinned package against the public vulnerability database.

```
pip-audit -r requirements.lock
```

151	0	exact	yes
packages pinned and checked	known vulnerabilities	versions (lock file)	re-checked before each release build

Reading the result. The comparison needs exact versions, which is the other reason for the lock file: what was audited is what the tests ran against and what the installers contain.

3.9 The quality gate: every commit, every build

Where	What runs	What happens on failure
On each commit (pre-commit)	Ruff, Ruff format, mypy strict, the fast tests	The commit is refused until fixed.
On each push (GitHub Actions CI)	Lint, types, tests with the coverage floor on Linux and macOS	The run is marked failed.
On each release build	PyInstaller + installer for both systems, then a smoke test of the installed app	No installer is published.

Note: the Windows test job on GitHub currently does not finish and is still being investigated; the Windows installer itself is built and smoke-tested.

4

Non-functional requirements

How the system behaves, beyond what it does

Each requirement below names the mechanism that meets it and, where possible, the measurement. **Met** means it is built and tested; **Met, with a gap** means a limit is stated.

4.1 Performance

30 students working at once	814/s calls handled per second	0 failed calls (also at 100 students)	~4 ms median to type, save or run a step
---------------------------------------	--	---	--

How it was measured. `scripts/benchmark.py` starts a throw-away server and has N students sign in and do a normal lab at the same time (15 edits per stage, run code, three stages, submit, open progress); every call is timed. 30 students made 1,770 calls in 2.2 s with none failing; 100 students made 5,900 calls in 7.4 s with none failing (typing median about 60 ms at that load). The event log writes about 38,000 events a second and running one piece of student code takes about 12 ms.

A real problem the benchmark found

The first 30-student run had calls taking 30 seconds and 17 failures. Cause: the database pool held only 15 connections and one request can hold two or three, so requests waited on each other. Fix: a pool of 50 (up to 200), a 30 s wait for the file lock, and a faster safe sync mode in WAL. After the fix: no failures and no call slower than 1.5 s. Two tests keep it that way. Full tables: `docs/PERFORMANCE.md`.

Area	Requirement	How it is met	Status
Performance	The screen never waits on the database	Event logger on its own thread, batched writes, SQLite WAL.	Met
Reliability	Student code cannot hang or crash the app	Every run is a separate process with a wall-clock limit; output is capped.	Met
Reliability	No work lost; clear failures	Autosave on every edit and on leaving a stage; events flushed before scoring and on shutdown; unexpected errors return a plain message and are logged.	Met
Reliability	No invented numbers	A signal with no evidence returns <i>none</i> with a reason (10 of 10 such cases checked).	Met
Logging	Problems can be investigated later	Structured JSON-lines log to a rotating file, plus an audit log of sign-ins, account changes and admin actions.	Met
Security	Passwords and secrets	Passwords: salted PBKDF2-SHA256, 200,000 rounds. Students' AI keys stay in the app's memory and never go to the server. The mail password is kept on the server and never shown back.	Met, with a gap*
Security	Access control and input	Roles enforced on the server; only approved emails can register; sign-in throttling; uploads limited by size and type; emailed codes expire in 15 minutes and allow 5 tries.	Met
Security	Isolation of untrusted code	Fresh subprocess per run; CPU limit everywhere it exists, memory limit on Linux.	Met
Privacy	Collect only what is needed	Students identified by approved email and roll number; AI keys never stored; progress visible to the student and their own professor.	Met

Area	Requirement	How it is met	Status
Maintainability	Typed, linted, tested, small modules	mypy strict, Ruff, coverage floor, pre-commit, CI; 8 focused modules.	Met, with a gap**
Portability	macOS and Windows	One code base; two installers built and smoke-tested by Actions; per-OS data folders and lab-mode lock-down.	Met
Usability	People can set it up	Guides in docs/, a one-click admin-password script, emailed first logins, plain-language messages.	Met

*Traffic is plain HTTP on the local network, the installers are not code-signed, the mail password is stored as text in the server's database, and lab mode cannot block every operating-system escape (for example Ctrl+Alt+Del). **api/bridge.py is too large.

5

Does the scoring measure what it claims?

Separate from the unit tests

Generic tests show the code runs. They do not show that each of the fourteen signals measures the behaviour it is defined to measure. A separate suite in `validation/` answers that, using the real `CavyApi` against a throw-away database, and the real local model for the three signals that need a language model.

53 / 53 known-answer trials match exactly	10 / 10 no-evidence cases return none	88 / 88 recomputations identical	14 / 14 signals implemented
---	---	--	---

Check	Result	What it means
Known-answer trials	53 / 53 exact	Each formula equals the value worked out by hand.
No-evidence trials	10 / 10 none	A signal with no evidence says so instead of inventing a number.
Repeatability	88 / 88 identical	Recomputing the same session five times gives the same output.
S1.1 help-seeking (model)	$r = 0.866, p = .005$	Rank agreement with hand-labelled examples.
S1.2 grounding (model)	$r = 0.843, p = .009$	Same check.
S3.1 understanding (model)	$r = 0.845, p = .034$	Same check; the ICC of 0.68 is not significant.

What this does not show
It shows the signals are computed correctly and repeatably. It does not show they predict real learning; that needs a study with students. One model-scored case is known to be wrong: a line-by-line restatement of the code was given full marks for understanding. Full write-up: `validation/report/VALIDATION_REPORT.pdf`.

6

Growth since the first audit

Same checks, larger system

The first audit was done on 20 September 2026 and covered the single-user desktop app (3,188 lines, 21 files, 145 tests). The current audit was done on 23 September 2026; in between, CAVY became a classroom system, and every check above was run again on the larger code.

	First audit (20 Sep 2026)	Current audit (23 Sep 2026)
Python source lines	3,188	9,825 in 44 files
Tests	144 passed	457 passed
Line + branch coverage	90.3%	89.0%
Mutation score (scoring engine)	63.7% (647 of 1,016)	83.3% (876 of 1051)
Ruff / mypy strict / Bandit	0 / 0 / 0	0 / 0 / 0
Dependencies	3 declared	151 pinned and audited
What it is	One desktop app per student	Class server, admin panel, professors, courses, lab mode, email

New in this period: the central server and admin panel, courses with levels and departments, professors and many-to-many classes, lab mode (full screen, key lock, focus rule, paste detection), first-login and password-reset emails, Groq as a full provider, structured logging, and the load benchmark. Each has tests.

7

Reproducing every number

One command each

Check	Command
Install	<code>pip install -r requirements.lock && pip install -e .</code>
Lint and types	<code>ruff check src tests ; mypy src</code>
Complexity	<code>radon cc src/eaal_platform -a -s ; radon mi src/eaal_platform -s</code>
Size and Halstead	<code>radon raw src/eaal_platform -s ; radon hal src/eaal_platform</code>
Tests, coverage	<code>pytest --cov=eaal_platform --cov-branch</code>
Security	<code>bandit -r src ; pip-audit -r requirements.lock</code>
Mutation	<code>mutmut run</code> (scoped to <code>signals/compute.py</code> in <code>pyproject.toml</code>)
Performance	<code>python scripts/benchmark.py --students 30</code>
Signal validation	<code>python validation/run_synthetic_suite.py</code>
This report	<code>python scripts/make_audit_report.py</code>

8

Overall assessment

What is strong, what is next

Section	Assessment
Codebase	Clear packages around one event-sourced database; one bridge class as the front-end boundary; reproducible install. Weak spot: <code>bridge.py</code> is too big.
Frameworks	Each choice tied to a constraint and compared with alternatives; few dependencies, all pinned. The comparison is reasoning and our own tests, not published benchmarks.
Code quality	Ruff, mypy strict and Bandit clean; 89% coverage; no function above 13 complexity; mutation score 83% on the scoring engine.
NFRs	Performance measured, logging added, security basics in place. HTTPS, signing and validation on a real class are still open.

Next steps, in order

1) Split `api/bridge.py` into modules. 2) HTTPS for the classroom server. 3) Code-sign the installers. 4) Run mutation testing on more than the scoring engine. 5) Test lab mode on more computers. 6) Run the scoring on a real class.

Report prepared by Group 04: Viral Kariya, Vanshika Ahuja, Yashika Parmar, Carol Maria.